A Regression-Guided Superoptimizer for a Reduced RISC-V ISA

1.Name: Pratyush Nandi
2. Registration Number: 24BIT0282

3. Dataset Description and Sample Data

This project does not use one downloaded, pre-existing dataset. It constructs a dataset from four sources and uses the project's Bundling-simulator to generate cycle-count labels. All counts below are planned sizes, not measured ones, until generation is run.

3.1 Link of the Dataset

There is no single public dataset link for the complete dataset because it is generated by this project.

Source code repository :

https://github.com/nandiprh/SuperOptimizer-Reduced_RISC-V

Other sources: generated random sequences and MCMC trajectories are project-generated; toy benchmarks are hand-written; Hacker’s Delight benchmarks are published benchmark programs; compiler-derived blocks are produced using GCC or Clang.

3.2 Dataset Description

The dataset contains short, loop-free, register-only basic blocks for a reduced RV32I instruction subset. Each row represents one instruction sequence. Sequence length is planned to range from 1 to 16 instructions. The training set is planned to contain approximately 50,000–200,000 labelled rows across all sources.

Source	Description	Planned share	Label / purpose
A — Randomly generated	Valid random RV32I sequences generated by the project's C++ tool; varied lengths and dependency structures.	About 40%	Simulator cycle counts; broad training coverage.
B — Benchmarks and mutations	Toy optimization targets and eligible Hacker’s Delight programs; mutated copies	About 30%	Targets plus labelled mutated sequences.

	add realistic patterns.		
C — Compiler-derived	Eligible basic blocks extracted from small C functions compiled with GCC or Clang for RV32I.	About 15%	Cycle labels; comparison baseline. Actual count unknown until extraction.
D — MCMC trajectories	Candidate sequences encountered during search, sampled and relabelled.	About 15%	Cycle labels; adapts training data to search behaviour.

Features include instruction count, opcode/class histograms, dependency depth, independent roots, average dependency distance, peak live-register count, WAW/WAR dependences, and estimated width (instruction count divided by dependency depth). The target variables are cycles_dynamic and cycles_bundled, produced by the simulator. The primary regression target will be one selected simulator mode.

Class label: this is a regression dataset, not a classification dataset. Its target is numeric cycle count, so it has no discrete class labels in the usual classification sense.

Randomly Generated Instruction Sequences (Source A)

Source: generated by the project's own C++ tool (tools/gen_dataset.cpp) over the v1 instruction set defined in the RISC-V unprivileged specification. Sequences are valid but not meaningful; the cost model never needs them to compute anything useful. Open source (own code).

Benchmark Programs: Toy Set and Hacker's Delight (Source B)

Source: a hand-written toy set (cancelling operations, double negation, shift/mask idioms), plus the 25-program Hacker's Delight benchmark suite used in STOKE , which originates in the loop-free program synthesis work of Gulwani et al. and the book by Warren . These programs serve two roles: they are the optimization targets, and, after random mutation, they are training data. Open source / published.

Compiler-Derived Basic Blocks (Source C)

Source: small C functions compiled with GCC or Clang for RV32I (-O0 and -O2, -march=rv32i, -mabi=ilp32); basic blocks extracted from the assembly output. This mirrors how Bansal and Aiken sourced code fragments. Open source toolchains.

MCMC Trajectory Samples (Source D)

Source: sequences visited during the project's own MCMC runs, logged and relabeled. The search mostly visits incorrect, unusual programs, so a model trained only on tidy sequences can mispredict where it is used. These samples addresses that distribution shift. Own data

Label Source: Bundling-simulator

Source: My own C simulator comparing dynamic scheduling (Tomasulo with reorder buffer and reservation stations) against static bundling (DAG-based dependency analysis with height-ranked list scheduling) for a stripped-down RV32I ISA . It replaces hardware measurement , playing the role BHive plays for Ithemal . Open source (GPL-3.0).

Sample Data Format and Class Labels

sample_path (illustrative)	source	kind	length	label
bench/toy/cancel_add.sopt	Toy target	target	3	reference
bench/hd/p01_turn_off_rightmost_1.sopt	Hacker's Delight	target	short	reference
data/random/seq_000123	A: random	training	1 to 16	cycles_dynamic, cycles_bundled
data/mutated/hd_p01_m07	B: mutated benchmark	training	1 to 16	cycles_dynamic, cycles_bundled
data/compiler/abs_O2_bb1	C: compiler-derived	training	1 to 16	cycles_dynamic, cycles_bundled
data/mcmc/chain03_iter0042000	D: MCMC sample	training	1 to 16	cycles_dynamic, cycles_bundled

V1 Instruction set (20 opcodes plus NOP)

Class	Opcodes	Semantics
R-type arithmetic	`ADD, SUB`	Wraparound add and subtract.
R-type logic	`AND, OR, XOR`	Bitwise operations.
R-type shift	`SLL, SRL, SRA`	Shift amount is the low 5 bits of rs2. SRA is an arithmetic shift.
R-type compare	`SLT, SLTU`	Result is 0 or 1; SLT signed, SLTU unsigned.
I-type arithmetic	`ADDI`	12-bit signed immediate, sign-extended.
I-type logic	`ANDI, ORI, XORI`	12-bit immediate sign-extended, so XORI rd, rs, -1 is bitwise NOT.
I-type shift	`SLLI, SRLI, SRAI`	Shift amount 0 to 31.
I-type compare	`SLTI, SLTIU`	Immediate sign-extended first; SLTIU then compares unsigned (SLTIU rd, rs, 1 tests rs == 0).
U-type	`LUI`	rd = imm20 shifted left by 12. Pure function of the immediate.
Internal	`NOP`	Fixed-length buffer filler; not counted in instruction count.

Register x0 always reads as zero and discards writes. x0 as a destination acts as a no-op, which the mutator can use to delete an instruction.

Instructions deliberately excluded from v1

Excluded	Reason
`AUIPC`	Reads the program counter, which has no stable meaning for a block searched out of context; equivalence would become address-dependent.
`BEQ, BNE, BLT, BGE, BLTU, BGEU, JAL, JALR`	Basic blocks contain no internal control flow.
`LB, LH, LW, LBU, LHU, SB, SH, SW`	Would require memory-state and aliasing equivalence checks. Deferred to v2.
`MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU`	M extension; excluded by project scope. Also removes the only multi-cycle ALU operations.
`FENCE, ECALL, EBREAK, CSR instructions`	Not computational.
`Floating-point extensions (F, D, Q)`	Excluded by project scope.

3.3 Sample Data — Two Illustrative Samples per Source

The examples below illustrate the four data sources using examples described in the project proposal. Cycle-count fields are generated by simulator

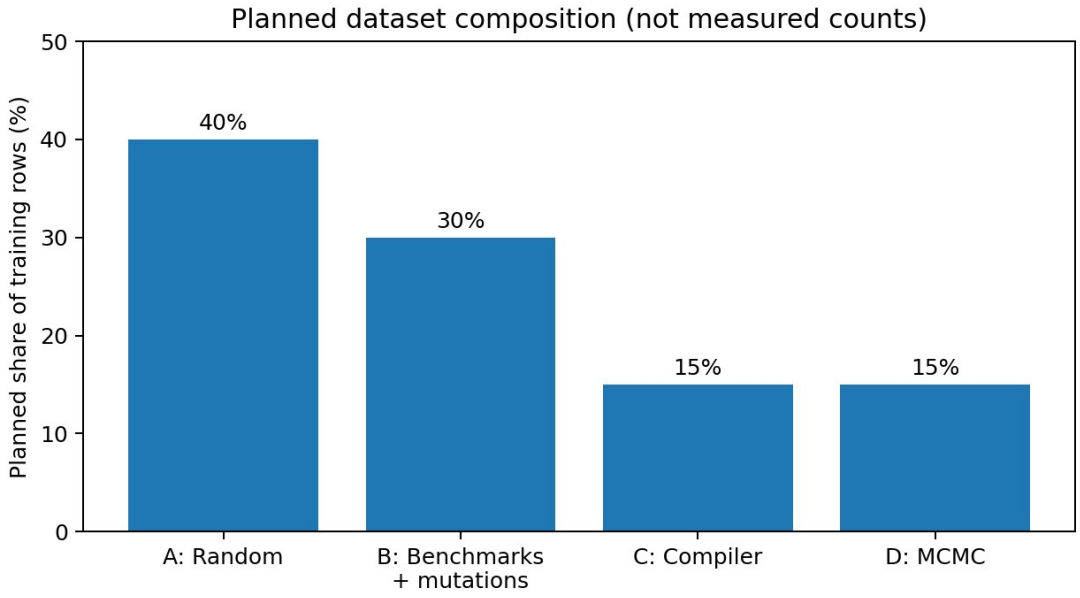
Source	Sample	Instruction sequence / description	Features shown	Cycle label
A — Random	A1	xor t0,a0,a1; slli t1,t0,3; sltu t2,t1,a0; addi t2,t2,-1	n_insns=4; dep_depth=4; roots=1	Simulator output pending
A — Random	A2	add t0,a0,a1; xor t1,a0,a1; and t2,a0,a1; or t3,a0,a1	n_insns=4; dep_depth=1; roots=4	Simulator output pending
B — Benchmark	B1	Target: add t1,a0,a1; sub t2,t1,a1; slli t3,t2,1	3 instructions; live-in a0,a1; live-out t3	Target/ reference; no training label required
B — Benchmark	B2	Expected optimized form: add t3,a0,a0	1 instruction; equivalent output intended	Simulator output pending for measured cycles
C — Compiler	C1	abs(x) idiom: srai t0,a0,imm; xor t1,a0,t0; sub a0,t1,t0	3 instructions in illustrative example	Simulator output pending
C — Compiler	C2	A second eligible compiler-derived RV32I block	Extracted from GCC/Clang output after filtering	Must be populated after extraction
D — MCMC	D1	Candidate sequence recorded from a search chain	chain_id, iteration, accepted, correctness_cost	Simulator output pending
D — MCMC	D2	Another candidate from a separate iteration/chain	Same feature schema as other sequences	Simulator output pending

3.4 Data Imbalance and Dataset Size Tables

A class-imbalance analysis is not applicable in the standard classification sense because the target is numeric cycle count rather than a class label. The proposal also gives percentages but not measured row counts. For a planned total of N rows, the approximate source-wise counts would be:

Source	Planned proportion	Approximate planned rows
A — Random	40%	0.40 × N
B — Benchmarks/mutations	30%	0.30 × N
C — Compiler-derived	15%	0.15 × N
D — MCMC trajectories	15%	0.15 × N
Total	100%	N = 50,000–200,000 planned

The following figures compare the proposal's planned source mixture with a hypothetical equal-source mixture. They are not evidence of class imbalance, and the equal-source chart is only an illustration(hypothetical)



Sample Dataset Instance

The feature values below were computed by code from the listings, not typed in. The cycle column is empty because it will be filled by the Bundling-simulator, and no figure is invented here.

→ A target and its expected optimization

```
target (3 instructions)              expected superoptimized output (1 instruction)
  add  t1, a0, a1     # a + b            add  t3, a0, a0     # a + a = 2a
  sub  t2, t1, a1     # (a + b) - b
  slli t3, t2, 1      # a << 1
live_in: a0, a1       live_out: t3
```

Equivalence rests on (a + b) - b = a, so a << 1 = a + a. Search is not told this identity; it finds the shorter form because correctness cost stays at zero while instruction count drops from 3 to 1. This is also the project's smoke test.

ID	Source	Sequence	n_insns	dep_depth	roots	cycles

S1	Toy target	`add t1,a0,a1`
		`sub t2,t1,a1`
		`slli t3,t2,imm`
			3	3	1	from simulator

S2	Optimized form of S1	`add t3,a0,a0`	1	1	1	from simulator

S3	C: abs(x) idiom	`srai t0,a0,imm`
		`xor t1,a0,t0`
		`sub a0,t1,t0`
			3	3	1	from simulator

S4	A: random	`xor t0,a0,a1`
		`slli t1,t0,imm`
		`sltu t2,t1,a0`
		`addi t2,t2,imm`
			4	4	1	from simulator

S5	A: fully parallel	`add t0,a0,a1`
		`xor t1,a0,a1`
		`and t2,a0,a1`
		`or t3,a0,a1`
			4	1	4	from simulator

S6	A: fully serial chain	`addi t0,a0,imm`
		`addi t0,t0,imm`
		`addi t0,t0,imm`
		`addi t0,t0,imm`
			4	4	1	from simulator

Example training row

```
seq_id          = 000123
source          = A
length          = 4
instructions    = "xor t0,a0,a1; slli t1,t0,3; sltu t2,t1,a0; addi t2,t2,-1"
n_insns         = 4        dep_depth = 4      roots = 1
hist_arith/logic/shift/cmp/lui = 1 / 1 / 1 / 1 / 0
cycles_dynamic  = <from simulator>
cycles_bundled  = <from simulator>
config_id       = cfg0
```

4. Methodology

4.1 Steps Involved in the Proposed Methodology

1. **Define the target program:** Select a loop-free basic block from the reduced RV32I subset and specify its live-in and live-out registers. The v1 scope excludes memory operations, branches, floating point, and multiply/divide instructions.
2. **Generate and collect sequences:** Create random sequences, collect toy/Hacker's Delight targets and mutations, extract eligible compiler-generated blocks, and record candidate sequences visited during MCMC runs.
3. **Filter and normalize the dataset:** Reject sequences containing unsupported instructions, expand pseudo-instructions into base instructions, remove duplicates after canonical register renaming, and preserve program-family grouping for train/validation/test splits.
4. **Extract features:** Compute instruction count, opcode/class histograms, dependency depth, independent roots, average dependency distance, peak live registers, WAW/WAR dependences, and estimated width using the same C++ feature function during training and inference.
5. **Generate cycle labels:** Run each labelled sequence through the Bundling-simulator. Record cycles_dynamic and cycles_bundled together with simulator configuration metadata. Run sanity tests on the simulator before bulk label generation.
6. **Train and evaluate the regression model:** Train gradient-boosted regression trees offline using the feature vectors and selected cycle target. Evaluate MAE, MAPE, Spearman rank correlation, and pairwise ordering accuracy on held-out program families and a separate MCMC-trajectory test set. Compare against instruction-count and linear-regression baselines.
7. **Search for equivalent programs:** Use Metropolis–Hastings MCMC to mutate instruction sequences. Evaluate candidates using a combined cost of correctness distance and performance; initially use instruction count, then use predicted cycle cost from the regression model.

8. **Check correctness:** Run candidate programs on test inputs and compare live-out registers with the target. Where available, encode both programs as 32-bit bit-vector formulas and use Z3 to prove equivalence. Add solver counterexamples to the test set and resume search.
9. **Compare and report results:** Keep verified candidates that reduce cost. Compare the final instruction count and simulated cycles against the original target/compiler output. For stochastic search, report results across multiple seeds rather than only the best run.

4.2 Detailed Explanation

The workflow combines a data-generation pipeline, a learned performance estimator, stochastic program search, and formal verification. The simulator provides the cycle-count labels used to train the regression model. The model is then called during the search loop to estimate candidate performance more cheaply than repeatedly invoking the simulator.

The MCMC search proposes local edits such as changing an opcode or operand, swapping instructions, or replacing an instruction. A correctness-distance term discourages behaviour that differs from the target, while the performance term favours shorter or faster sequences. Occasionally accepting worse candidates helps the search escape local minima. Search results are not automatically trusted: test cases filter candidates, and Z3 is used where possible to prove equivalence exactly.

The dataset split must be made by benchmark/program family rather than by individual row so that mutated versions of the same target do not leak across training and test sets. A separate untouched MCMC-trajectory test set checks whether the regression model remains accurate on sequences similar to those encountered during actual search.

Algorithms Used

The system combines five algorithms. Two search for programs, one verifies them, one produces the cost labels, and one learns the cost from those labels

#	Algorithm	Role in the system	Category
1	Metropolis-Hastings MCMC	Main stochastic search over instruction sequences	Search
2	Enumerative search	Exhaustive baseline for short sequences; ground truth for algorithm 1	Search
3	Dependency DAG, list scheduling and Tomasulo	Cost oracle: produces cycle labels; source of dependency features	Scheduling / simulation
4	Gradient-boosted regression trees	Learned cost model that replaces simulator calls in the search loop	Learning
5	SMT equivalence checking with counterexample loop	Proves a found sequence equals the target; feeds failures back to the tests	Verification

Algorithm 1: Metropolis-Hastings MCMC search

Purpose. Find a low-cost program equivalent to the target without enumerating the space, which is far too large beyond a few instructions. The search is a random walk that proposes a local change, evaluates its cost, and accepts it with a probability that favours improvements but occasionally allows worse programs, so it can leave local minima. With a symmetric proposal, the chain's long-run distribution is proportional to exp(-beta * cost), so it spends most of its time near cheap programs

Algorithm 2: Enumerative search

Purpose. Exhaustively try every sequence of length 1, 2, ..., k over the register and immediate pools, keep those that pass a small test set, then verify the survivors. This is the classic approach introduced by Massalin and scaled with hybrid testing and solver verification by Bansal and Aiken. Here it serves as a baseline, if MCMC cannot match the enumerated optimum on short targets, the search is wrong.

Algorithm 3: Dependency DAG, list scheduling and Tomasulo (cost oracle)

Purpose. Produce the cycle-count labels and the dependency features. The Bundling-simulator builds a dependency DAG from register def-use chains, then computes cost two ways. Static bundling uses height-ranked list scheduling: each instruction is ranked by its longest path to the end of the block (critical-path priority, after Hu) and packed into issue bundles. Dynamic scheduling uses Tomasulo's algorithm with reservation stations and a reorder buffer, which issues out of order and commits in order.

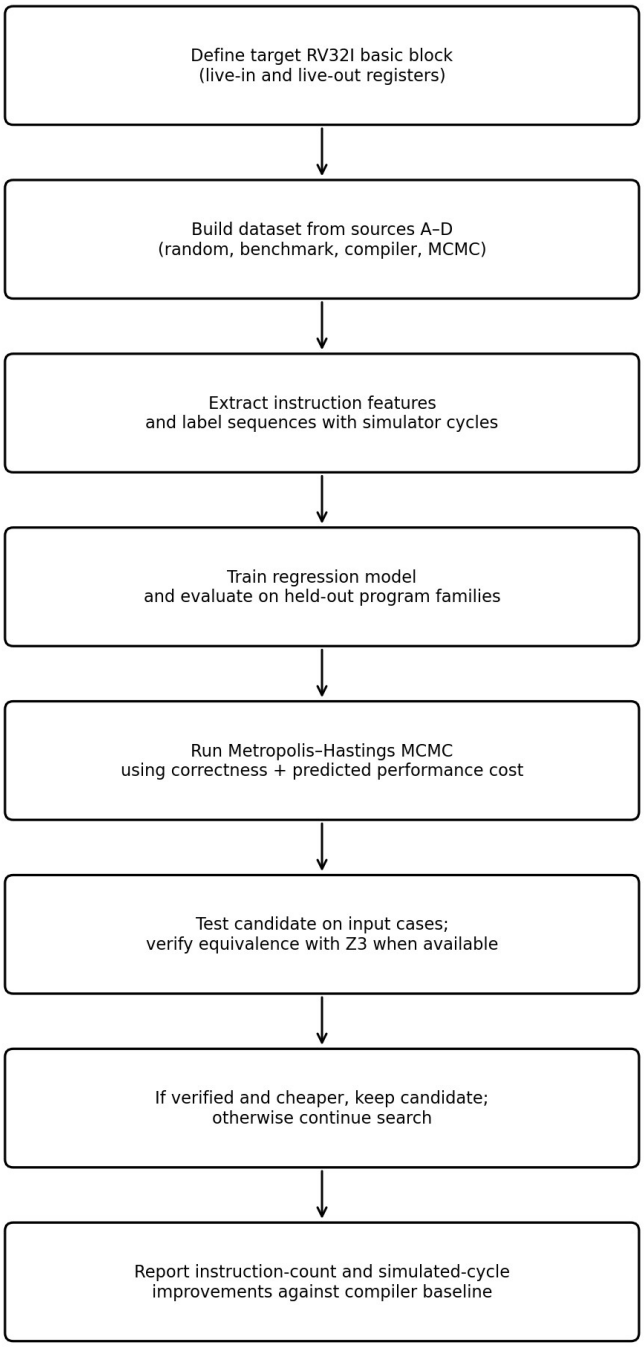
Algorithm 4: Gradient-boosted regression trees (learned cost model)

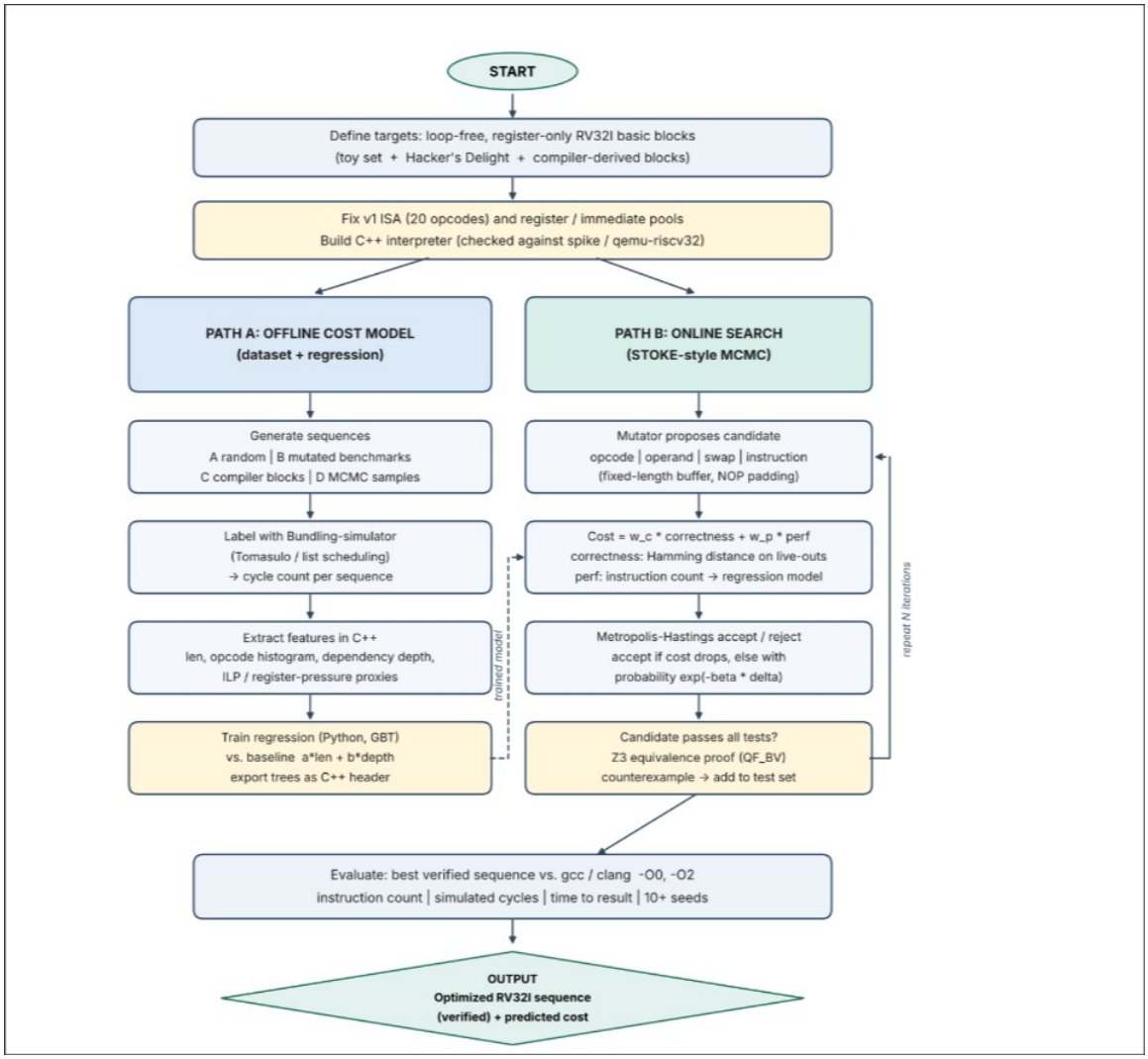
Purpose. Predict cycles from sequence features, so the search does not run the simulator at every step. Gradient boosting fits an additive ensemble of shallow regression trees, each trained on the residual error of the ensemble so far [14]. Tabular features that interact non-linearly, such as opcode mix and dependency depth, are the setting where boosted trees usually beat linear regression. Standard implementations are XGBoost , LightGBM and scikit-learn . This follows the idea of learned basic-block cost models such as Ithemal and the BHive benchmark , but with a far simpler model and self-generated labels.

Algorithm 5: SMT equivalence checking with a counterexample loop

Purpose. Test cases can pass a wrong program, so the final candidate is checked exactly. Both programs are encoded as 32-bit bitvector formulas (theory QF_BV) in the Z3 solver and the solver is asked whether any input makes a live-out register differ. If the answer is unsatisfiable, the programs are equivalent. If it is satisfiable, the model returned is a counterexample that is added to the test set and the search resumes, in the style of counterexample-guided inductive synthesis ; STOKE uses the same loop.

4.3 Methodology Flowchart




START
Define targets: loop-free, register-only RV32I basic blocks
(toy set + Hacker's Delight + compiler-derived blocks)
Fix v1 ISA (20 opcodes) and register / immediate pools
Build C++ interpreter (checked against spike / qemu-riscv32)
PATH A: OFFLINE COST MODEL
(dataset + regression)
PATH B: ONLINE SEARCH
(STOKE-style MCMC)
Generate sequences
A random | B mutated benchmarks
C compiler blocks | D MCMC samples
Label with Bundling-simulator
(Tomasulo / list scheduling)
→ cycle count per sequence
Extract features in C++
len, opcode histogram, dependency depth,
ILP / register-pressure proxies
Train regression (Python, GBT)
vs. baseline a*len + b*depth
export trees as C++ header
trained model
Mutator proposes candidate
opcode | operand | swap | instruction
(fixed-length buffer, NOP padding)
Cost = w_c * correctness + w_p * perf
correctness: Hamming distance on live-outs
perf: instruction count → regression model
Metropolis-Hastings accept / reject
accept if cost drops, else with
probability exp(-beta * delta)
Candidate passes all tests?
Z3 equivalence proof (QF_BV)
counterexample → add to test set
repeat N iterations
Evaluate: best verified sequence vs. gcc / clang -O0, -O2
instruction count | simulated cycles | time to result | 10+ seeds
OUTPUT
Optimized RV32I sequence
(verified) + predicted cost